

CLAUDE.md — Pour Decisions: Cocktail Recommender

Instruction file for Claude Code. Read this in full before writing any code.

Project Overview

Pour Decisions: Cocktail Recommender is a local Python CLI application that recommends cocktails based on free-text user input — moods, vibes, ingredients, or any natural language description. The core engine uses sentence embeddings and cosine similarity search against a cached cocktail dataset from TheCocktailDB.

This is a portfolio project. Code should be clean, well-commented, and reflect sound data science and software engineering principles.

Goals and Non-Goals

Goals

- Semantic search: a query like "tropical and refreshing" should surface coconut/lime/rum cocktails
- Fully local after initial data fetch — no API calls at query time
- Fast query response (under 1 second)
- Polished CLI experience using `rich` for formatted output
- Reproducible index build step

Non-Goals

- No generative LLM at inference time (embeddings + similarity only)
- No web UI (Streamlit is a stretch goal only, do not build unless explicitly asked)
- No user accounts, persistence, or logging

Tech Stack

Layer	Library	Notes
Data fetching	<code>requests</code>	Loops a-z against TheCocktailDB free API
Data handling	<code>pandas</code>	Cocktail records and composite doc construction
Embeddings	<code>sentence-transformers</code>	Model: <code>all-MiniLM-L6-v2</code>
Similarity search	<code>sklearn</code> (<code>cosine_similarity</code>)	Dataset is small; FAISS is not needed
Index persistence	<code>numpy</code> , <code>joblib</code>	Embeddings saved as <code>.npy</code> ; metadata as <code>.pkl</code>

Layer	Library	Notes
CLI	<code>typer</code>	Single-query and interactive REPL modes
Output formatting	<code>rich</code>	Panels, tables, colored text

All dependencies must be declared in `requirements.txt`. Do not hardcode anything that belongs in configuration — use constants at the top of each module or a `config.py`.

Project Structure

```

cocktail-recommender/
├── CLAUDE.md
├── README.md
├── requirements.txt
├── config.py                # Paths, model name, top-k default, API base
├── URL
├── data/
│   └── fetch.py            # Fetches and caches cocktail data from
├── TheCocktailDB
│   ├── cache/              # Auto-created; stores raw_cocktails.json
│   ├── embeddings/
│   │   ├── build_index.py  # Builds and saves embedding index
│   │   └── index/          # Auto-created; stores embeddings.npy and
├── metadata.pkl
├── recommender/
│   └── search.py           # Query embedding, similarity search, result
├── formatting
├── cli.py                  # Typer entrypoint – interactive and single-
└── query modes

```

All `cache/` and `index/` directories should be created programmatically if they do not exist. Do not assume they are pre-created.

Roadmap

Phase 1 — Project Setup

- ☐ Initialize repo with this structure
- ☐ Create `requirements.txt` with all dependencies
- ☐ Create `config.py` with all path constants, model name, API URL, and default top-k

Phase 2 — Data Pipeline (`data/fetch.py`)

- ☐ Fetch all cocktails from TheCocktailDB by looping letters a-z using the `/search.php?f={letter}` endpoint
- ☐ Deduplicate on `idDrink`

- ☐ Normalize ingredient fields (TheCocktailDB uses `strIngredient1` through `strIngredient15`) into a single clean list, dropping nulls
- ☐ Save to `data/cache/raw_cocktails.json`
- ☐ Skip fetch if cache already exists unless `--force` flag is passed
- ☐ Log fetch progress with `rich` progress bar

Phase 3 — Embedding Index (`embeddings/build_index.py`)

- ☐ Load from `data/cache/raw_cocktails.json`
- ☐ Build composite document string per cocktail: `"{name}. A {category} served in a {glass}. Made with {ingredients}."`
- ☐ Embed all documents using `sentence-transformers` (`all-MiniLM-L6-v2`)
- ☐ Save embeddings to `embeddings/index/embeddings.npy`
- ☐ Save cocktail metadata (name, category, glass, ingredients, instructions) to `embeddings/index/metadata.pkl` as a list of dicts
- ☐ Skip build if index already exists unless `--force` flag is passed
- ☐ Log build progress with `rich`

Phase 4 — Search Engine (`recommender/search.py`)

- ☐ Load embeddings and metadata on module import (lazy-load on first call)
- ☐ Expose a `search(query: str, top_k: int) -> list[dict]` function
- ☐ Embed the query string using the same model
- ☐ Compute cosine similarity between query embedding and all cocktail embeddings
- ☐ Return top-k results sorted by descending similarity score
- ☐ Each result dict should include: name, category, glass, ingredients, similarity score, and instructions

Phase 5 — CLI (`cli.py`)

- ☐ Default command launches interactive REPL loop — prompt user for input, display results, repeat until `exit` or `quit`
- ☐ `--query` flag accepts a single query string for non-interactive use
- ☐ `--top-k` flag (default from `config.py`) controls number of results
- ☐ Each result rendered as a `rich` Panel containing: cocktail name (title), similarity score, category, glass, ingredients list, and instructions
- ☐ Graceful handling of empty results or index-not-found errors

Phase 6 — Polish

- ☐ Write `README.md` titled "Pour Decisions: Cocktail Recommender" covering: project purpose, setup instructions, example queries, and a brief explanation of why embeddings were chosen over keyword search
- ☐ Verify all modules have descriptive present-participle comments
- ☐ Smoke test with at least 5 diverse query types: ingredient-based, mood-based, spirit-based, occasion-based, flavor-based

Code Conventions

These apply to every file written in this project.

Comments Write all comment lines in present participle form.

```
# Fetching cocktail data from TheCocktailDB
# Normalizing ingredient fields into a flat list
# Computing cosine similarity against indexed embeddings
```

No hardcoded values All paths, model names, URLs, and numeric defaults live in `config.py`. Import from there — never inline a string path or magic number in logic code.

Libraries over manual implementation Use `pandas` for tabular operations, `sklearn` for similarity, `sentence-transformers` for embedding. Do not reimplement anything a well-maintained library already handles.

Error handling Raise informative errors if the cache or index is missing when a downstream step expects it. Guide the user toward the correct command to fix the issue (e.g., "Run `python data/fetch.py` first").

Type hints Use type hints on all function signatures.

Key Architectural Decisions

Why embeddings and not keyword search Keyword search fails the core use case. A query like "something cozy for a rainy night" contains no ingredient names, yet it should semantically surface warm, spirit-forward cocktails. Sentence embeddings encode semantic meaning, closing the gap between natural language intent and structured recipe data.

Why `all-MiniLM-L6-v2` Approximately 80MB, runs fast on CPU, and produces high-quality semantic embeddings for short-to-medium texts. Well-suited for a local portfolio project with no GPU requirement.

Why composite document strings Transformers perform better on natural sentence structures than raw concatenated field values. Constructing a human-readable sentence from structured fields gives the embedding model richer context to encode.

Why `sklearn` over `FAISS` TheCocktailDB contains roughly 500 cocktails. At that scale, brute-force cosine similarity via `sklearn` completes in milliseconds. `FAISS` introduces complexity with no meaningful performance benefit here.

Running the Project

```
# Installing dependencies
pip install -r requirements.txt

# Fetching cocktail data
python data/fetch.py
```

```
# Building embedding index
python embeddings/build_index.py

# Launching interactive CLI
python cli.py

# Running a single query
python cli.py --query "something citrusy and refreshing" --top-k 5
```